

A Comparative Study of Reinforcement Learning in Spacecraft Orbit Control^{*}

Jialu Li^{*} Xuebo Yang^{*} Heng Yang^{*} Meiling Hu^{*}

^{*} Harbin Institute of Technology, Harbin, 150001, China(e-mails: jialuli@stu.hit.edu.cn(J.L. Li); xueboyang@hit.edu.cn(X.B. Yang); hengyang@stu.hit.edu.cn(H. Yang); mlhu@stu.hit.edu.cn(M.L. Hu);

Abstract: This paper addresses the close-range rendezvous in spacecraft on-orbit services, focusing on orbit control under perturbations like the non-spherical earth perturbation and controller noise. By using Deep Reinforcement Learning algorithms (Proximal Policy Optimization, Two-delay Deep Deterministic Policy Gradient, and Soft Actor-Critic), the simulations demonstrate superior that the method used in this paper performs well in terms of control accuracy, stability, and disturbance rejection capability. A detailed analysis of the merits and limitations of three algorithms is presented, ultimately concluding with reinforcement learning applications in autonomous space missions.

Copyright © 2025 The Authors. This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0/>)

Keywords: Spacecraft orbit control, deep reinforcement learning, Close-Range rendezvous, decision making and autonomy, control algorithms implementation.

1. INTRODUCTION

With advancing space technology and the outer space has emerged as the fourth strategic arena for major power competition, modern spacecraft need to undertake increasingly complex tasks like on-orbit servicing and close observation of satellites, where close-range rendezvous is essential. However, as a critical phase of on-orbit missions, guidance technology is still immature and warrants further research. This paper studies spacecraft tracking during close-range rendezvous, considering non-spherical Earth perturbations (Wang et al. (2023)), controller noise, and other nonlinear effects.

Reinforcement learning (RL), as a task policy optimization method to find the optimal action sequence to achieve the task goal through policy iteration, can make full use of historical information while maintaining great exploration capability. Representative algorithms (TRPO (Schulman et al. (2015)) et al.) have succeeded in domains like expert-level Go gaming (Silver et al. (2016)). Recent studies further demonstrate its potential for on-orbit services planning (Zhao et al. (2015)), such as spacecraft ignition control optimization. Deep reinforcement learning (Tang et al. (2017)) integrates the generalization capability of deep learning and excels in intelligent target capture (e.g., spacecraft escape games (Tao et al. (2022))), highlighting its effectiveness in autonomous spacecraft control.

However, existing studies predominantly address spacecraft attitude control (Yadava et al. (2018)), with limited focus on the orbit maneuver control. For nonlinear orbit transfer with high terminal accuracy requirements, Cheng et al. (2018) tested multiple RL algorithms (DQN, DDPG, TRPO and A3C) but ultimately adopted DNNs due to

suboptimal performance. While traditional methods (e.g., backstepping) offer robustness, their fixed parameters may compromise stability under uncertainties. Thus, this paper leverages Deep Reinforcement Learning (DRL) to enhance orbit terminal relative navigation precision during spacecraft close-range rendezvous.

The core contribution of this paper lies in addressing spacecraft orbit tracking problems of two spacecrafts under non-spherical Earth perturbations and controller noise by training three DRL algorithms. And this paper is organized as follows: Section I introduces the current research status of spacecraft tracking missions. Section II establishes the spacecraft orbital dynamics model and the related Reinforcement Learning environment with a brief overview of basic Reinforcement Learning algorithm. Section III details the logical structure of Proximal Policy Optimization (PPO), Two-delay Deep Deterministic Policy Gradient (TD3), and Soft Actor-Critic (SAC) algorithms. Section IV compares their control performance through simulations in accuracy, efficiency, and disturbance rejection capability. Section V outlines future work.

2. PROBLEM STATEMENT

2.1 Spacecraft Orbital Dynamics Equation

In this paper, we consider the target-chasing rigid spacecraft system as an example. As illustrated in Fig. 1, the motion of the spacecraft is described using three orthogonal coordinate systems: the Earth-Centered Inertial (ECI) frame $\mathcal{I}$, the chasing spacecraft body-axis coordinate system $\mathcal{B}$, and the target spacecraft body-axis coordinate system $\mathcal{T}$.

The gravitational field of a homogeneous sphere equals that of a point mass at its center. However, due to Earth's non-spherical shape, spacecraft in rendezvous missions

^{*} This work was supported in part by the National Natural Science Foundation of China (NO.62373130). Corresponding author: Xuebo Yang.

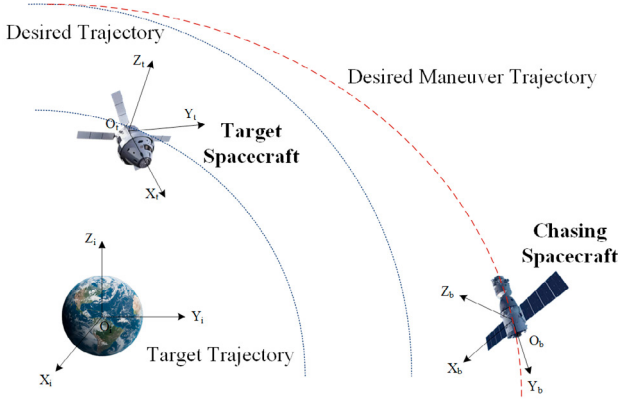


Fig. 1. Spacecraft related coordinate systems schematic

experience perturbation forces. This paper focuses on the non-spherical earth perturbation (J_2 perturbation), denoted as $\mathbf{a}_{J2}$, with its expression given by:

$$\mathbf{a}_{J2} = -\frac{3\mu J_2 R_e^2}{2R^7} \begin{bmatrix} X(R^2 - 5Z^2) \\ Y(R^2 - 5Z^2) \\ Z(3R^2 - 5Z^2) \end{bmatrix}, \quad (1)$$

where $R = \sqrt{X^2 + Y^2 + Z^2}$ is the geocentric distance of the satellite. Considering the studied spacecraft is a nonlinear rigid body in an elliptical orbit, its dynamics equations in the ECI frame can be expressed as:

$$\ddot{\mathbf{r}}_b = -\mu \frac{\mathbf{r}_b}{r_b^3} + \mathbf{a}_{J2} + \frac{\mathbf{F}_b}{m}, \quad (2)$$

$$\ddot{\mathbf{r}}_t = -\mu \frac{\mathbf{r}_t}{r_t^3} + \mathbf{a}_{J2}, \quad (3)$$

where $\mathbf{r}_b \in \mathbb{R}^3$ and $\mathbf{r}_t \in \mathbb{R}^3$ represent the position vectors of the chasing spacecraft and the target spacecraft in the ECI frame respectively, and $\mu = 398600.4418 \text{ km}^3/\text{s}^2$ is the Earth gravitational constant, and $\mathbf{F}_b \in \mathbb{R}^3$ denotes the chasing spacecraft control moment.

Define the relative position vector $\mathbf{r}$ and the relative velocity vector $\mathbf{v}$ in the ECI frame as:

$$\mathbf{r} = \mathbf{r}_b - \mathbf{r}_t \quad (4)$$

$$\mathbf{v} = \dot{\mathbf{r}} = \dot{\mathbf{r}}_b - \dot{\mathbf{r}}_t \quad (5)$$

2.2 Markov Decision Process

Reinforcement Learning (RL) is a machine learning method that learns the optimal policy through the interaction between the agent and the environment. Markov Decision Process (MDP) provides a strictly mathematical framework for RL (Sutton and Barto (1999)). The spacecraft orbit control problem can be modeled as the MDP in continuous state-action space. In this paper, the spacecraft orbit control problem is formalized as a multi-dimensional continuous space MDP six-tuple $(\mathcal{S}, \mathcal{A}, \mathcal{G}, \mathcal{P}, \mathcal{R}, \gamma)$ with the following specific environment sets:

- (1) The state space is

$$\mathcal{S} = (\mathbf{r}, \mathbf{v}) = ([r_x, r_y, r_z], [v_x, v_y, v_z]). \quad (6)$$

- (2) The action space is

$$\mathcal{A} = (\mathbf{F}_b) \doteq ([f_x, f_y, f_z]). \quad (7)$$

- (3) The objective is

$$\mathcal{G} = (\mathbf{r}_g, \mathbf{v}_g) \doteq ([r_{gx}, r_{gy}, r_{gz}], [v_{gx}, v_{gy}, v_{gz}]). \quad (8)$$

- (4) The state transition probability is $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$.
 (5) The reward function is $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{R}$.
 (6) γ is the discount factor.

During the training process, the agent observes the environment state $s_t \in \mathcal{S}$ and selects the action $a_t \in \mathcal{A}$ to execute based on the current policy $\pi(a | s)$, then the environment transfers to a new state according to the state transition probability $P(s_{t+1} | s_t, a_t) \in \mathcal{P}$ and returns the immediate reward $r_t = R(s_t, a_t) \in \mathcal{R}$. The agent maximizes the expected (discounted) cumulative reward $J(\pi_s) = \mathbb{E}[\sum \gamma^t r_t | s_t = s]$ by optimizing the policy π , while satisfying the environmental constraints, where $\gamma \in [0, 1]$ is the discount factor (Zhao (2025)).

Definition. The Actor-Critic (AC) algorithm (Barto et al. (1983)) is the framework basis of many current advanced algorithms such as PPO, DDPG, and other algorithms.

The Actor network learns the optimal policy parameter $\theta \in \Theta$ by maximizing the cumulative reward as follows:

$$\begin{aligned} \theta_{t+1} &= \theta_t + \alpha \nabla_{\theta} J(\pi_{\theta_t}) \\ &= \theta_t + \alpha \mathbb{E}_{S \sim \eta, A \sim \pi} [\nabla_{\theta} \ln \pi(A | S, \theta_t) Q_{\pi}(S, A)], \end{aligned} \quad (9)$$

where $\nabla_{\theta} J(\pi_{\theta_t})$ is the gradient of the policy objective function $J(\pi_{\theta})$, $\pi(A | S, \theta_t)$ is the parameterized policy function, $Q_{\pi}(S, A)$ is the state-action value function, α is the fixed network learning rate, and η is the state distribution.

The Critic network learns the state-action value function $Q_{\pi}(S, A)$ by minimizing the Temporal Difference (TD) error, with its loss function defined as:

$$\text{Loss} = (R(s_t, a_t) + \gamma Q_{\pi}(s_{t+1}, a_{t+1}) - Q_{\pi}(s_t, a_t))^2, \quad (10)$$

where $R(s_t, a_t)$ is the reward function and the target value function is $y = R(s_t, a_t) + \gamma Q_{\pi}(s_{t+1}, a_{t+1})$.

3. REINFORCEMENT LEARNING ALGORITHMS

3.1 Proximal Policy Optimization Algorithm

The AC algorithm updates parameters with fixed step sizes α , which may cause instability or slow convergence. The proximal policy optimization (PPO) algorithm (Schulman et al. (2017)) addresses this issue by modifying the optimization function (see Algorithm 1 below).

Denote the policy objective function of the policy π by $J(\pi)$ and the policy objective function of the policy $\tilde{\pi}$ by $J(\tilde{\pi})$. The state distribution of the old policy π is used to approximate the state distribution of the new policy $\tilde{\pi}$ as follows:

$$J(\tilde{\pi}) = J(\pi) + \sum_s \eta_{\pi}(s) \sum_a \tilde{\pi}(a | s) A^{\pi}(s, a), \quad (11)$$

where $\eta_{\pi}(s) = \sum \gamma^t P(s_t = s)$. The proof in Kakade and Langford (2002) shows that $J(\pi)$ can be optimized by optimizing the equation as follows:

$$\max \text{imize}_{\theta} [L_{\pi}(\tilde{\pi}) - CD_{KL}^{max}(\pi, \tilde{\pi})], \quad (12)$$

where $L_{\pi}(\tilde{\pi}) = \sum_s \eta_{\pi}(s) \sum_a \tilde{\pi}(a | s) A^{\pi}(s, a)$ and $C = \frac{2\varepsilon\gamma}{(1-\gamma)^2}$. Thus we obtain the following expected return

Algorithm 1 PPO algorithm

Input: Differentiable parameterized policy $\pi(a|s; \theta)$, differentiable parameterized state-value function $V(s; w)$;

Hyperparameters: $\alpha^\theta > 0$, $\alpha^w > 0$;

Initialize: Randomly initialize policy parameters θ , state-value function parameters w ;

Loop:

Initialize state s ;

Sample new trajectory:

For each step: sample data $\{s_t, a_t, r_t\}$ using current policy $a \sim \pi(\cdot|s; \theta)$;

If s_t is terminal state s_T or max horizon s_{mb} :

$\hat{Q}_t = V(s')$;

Else:

$$\hat{Q}_t = \sum_{t'=t}^{T'-1} \gamma^{t'-t} r_{t'} + \gamma^{T'-t} V(s_T; w);$$

Store current parameters θ as θ_{old} ;

Update parameters using mini-batch data:

$$\min_{\theta} \hat{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 \pm \epsilon) \left(\hat{Q}_t - V(s_t; w) \right) \right) \right],$$

$$\text{where } r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)};$$

$$w \leftarrow w + \frac{\alpha^w}{mT} \sum_{i=0}^m \sum_{j=0}^{T'} \left(Q_j^{(i)} - V_w(s_j^{(i)}) \right) \nabla_w V_w(s_j^{(i)}; w);$$

under the policy update constraint after replacing the action samples:

$$\sum_a \tilde{\pi}_{\theta}(a | s_n) A_{\theta_{old}}(s_n, a) = \mathbb{E}_{a \sim \pi} \left[\frac{\tilde{\pi}_{\theta}(a | s_n)}{\pi_{\theta}(a | s_n)} A_{\theta_{old}}(s_n, a) \right]. \quad (13)$$

To simplify the constraint, the sampling distribution is used to instead the discounted distribution (i.e. $s \sim \eta_{\theta_{old}} \rightarrow s \sim \pi_{\theta_{old}}$), we obtain the target expected return:

$$\max \text{imize}_{\theta} \mathbb{E}_{s \sim \eta_{\theta_{old}}, a \sim \pi_{\theta_{old}}} \left[\frac{\pi_{\theta}(a | s)}{\pi_{\theta_{old}}(a | s)} A_{\theta_{old}}(s, a) \right], \quad (14)$$

subject to $\mathbb{E}_{\pi_{\theta_{old}}} [D_{KL}^{max}(\pi_{\theta_{old}} \| \pi_{\theta})] \leq \delta$.

To enable large-scale optimization, the constraints are transformed into a simple ratio constraints of the old and new policies as $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$. This ratio is constrained to the vicinity of 1.

In summary, the objective function of PPO algorithm is

$$L(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (15)$$

where $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$.

3.2 Two-delay Deep Deterministic Policy Gradient Algorithm

Combined with deep neural networks, TP Lillicrap et al. proposed the Deep Deterministic Policy Gradient (DDPG) algorithm (Lillicrap et al. (2019)), based on the Deep Q-Network (DQN) algorithm and Deterministic Policy Gradient (DPG) algorithm within the AC framework. It uses deep neural networks $\mu'(s | \theta^{\mu'})$ and $Q'(s, a | \theta^{Q'})$ to respectively approximate the policy function $\mu(s | \theta^{\mu})$ and state-action value function $Q(s, a | \theta^Q)$, trained via the stochastic gradient descent algorithm for stability.

Unlike stochastic policies, the DDPG algorithm employs a deterministic policy μ mapping states s directly to a specific continuous actions $a = \mu(s | \theta^{\mu})$, making it more efficient than PPO algorithm for continuous action spaces by avoiding the computational overhead of high-dimensional sampling.

The two-delay deep deterministic policy gradient (TD3) algorithm (Fujimoto et al. (2018)) was proposed to address three key limitations of the DDPG algorithm: the overestimation of Q , high variance in network updates, and unstable policy updates. The TD3 algorithm employs two Critic networks Q_{θ_1} and Q_{θ_2} and uses the minimum of them when computing the target value function y , which helps mitigate the overestimation of Q .

By adding clipped noise $a' = \mu'(s') + \epsilon$ to the specific action, where $\epsilon \sim \text{clip}(\mathcal{N}(0, \sigma), -c, c)$, the TD3 algorithm addresses the instability issues (such as high sensitivity to approximation errors and large variance in network updates) in the DDPG algorithm caused by overfitting. Meanwhile this clipped noise ensures that excessive deviation from the original action is prevented.

Additionally, in the DDPG algorithm, the gradient of the Critic network guides the parameters of the Actor network to be updated through the chain rule. An unstable Critic network can cause oscillating Actor network updates or convergence to suboptimal policies. TD3 algorithm mitigates this issue by delaying the Actor network updates, i.e. updating the Critic network at every step while updating the Actor network only every d steps. This ensures the Critic network stabilizes first, thereby guaranteeing the stability of policy learning.

In summary, TD3 is more adapted to the continuous action space through the three major mechanisms of clipped double Q -Learning, target policy smoothing and delayed policy update. Algorithm 3 outlines the logical structure.

Algorithm 2 TD3 algorithm

Initialize: Critic networks Q_1, Q_2 ; Actor network π ; Target networks Q'_1, Q'_2, π' ; Replay buffer $\mathcal{R}$.

Hyperparameters: Critic networks parameters $\theta_{Q_1}, \theta_{Q_2}$; Actor network parameters θ_{π} ; Target networks parameters $\theta_{Q'_1} \leftarrow \theta_{Q_1}, \theta_{Q'_2} \leftarrow \theta_{Q_2}, \theta_{\pi'} \leftarrow \theta_{\pi}$.

Loop:

Select action with exploration noise: $a = \pi(s) + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \epsilon_{\text{noise}})$;

Execute a , observe reward r and new state s' ;

Store transition (s, a, r, s') in $\mathcal{R}$;

Update step:

Sample N transitions minibatch (s_i, a_i, r_i, s'_i) from $\mathcal{R}$;

$\tilde{a}'_i = \pi'(s'_i) + \epsilon, \epsilon \sim \text{clip}(\mathcal{N}(0, \sigma), -c, c)$;

$y_i = r_i + \gamma \cdot \min(Q'_1(s'_i, \tilde{a}'_i), Q'_2(s'_i, \tilde{a}'_i))$;

Update Critic networks:

Minimize loss of $Q_{j=1,2}$:

$$L_{j=1,2} = \frac{1}{N} \sum (y_i - Q_{j=1,2}(s_i, a_i))^2;$$

After d steps update Actor:

$$\nabla_{\theta_{\pi}} J(\theta_{\pi}) = \frac{1}{N} \sum \nabla_a Q_j(s_i, a) \big|_{a=\pi(s_i)} \nabla_{\theta_{\pi}} \pi(s_i);$$

Update target networks:

$$Q'_{j=1,2}: \theta_{Q'_{j=1,2}} \leftarrow \tau \theta_{Q_{j=1,2}} + (1 - \tau) \theta_{Q'_{j=1,2}};$$

$$\pi': \theta_{\pi'} \leftarrow \tau \theta_{\pi} + (1 - \tau) \theta_{\pi'};$$

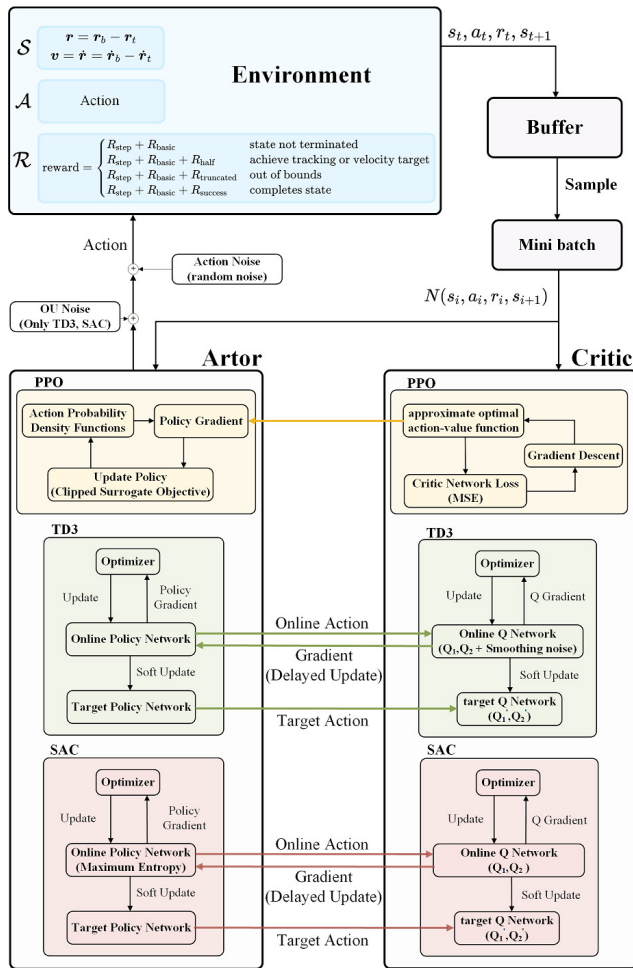


Fig. 2. Spacecraft orbit control RL training framework

3.3 Soft Actor-Critic Algorithm

The Soft Actor-Critic (SAC) (Haarnoja et al. (2019)) algorithm extends the TD3 framework for high-dimensional continuous spaces by incorporating maximum entropy policy (see Algorithm 3 below). Built on the Actor-Critic architecture, SAC algorithm augments the constraint of the policy objective function with an entropy regularization term as follows:

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t (r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot \mid s_t))) \right], \quad (16)$$

where $\mathcal{H}(\pi) = -\log \pi(a \mid s_t)$ is the policy entropy, and $\alpha > 0$ is a temperature coefficient to balance exploration (entropy) and exploitation (reward). By maximizing both return and entropy, SAC avoids premature convergence to suboptimal policies and thus enhances exploration in complex environments.

Based on the TD3 architecture, the SAC algorithm performs better than PPO in high-dimensional continuous action spaces through its stochastic policy, the deep neural network, and two Critic networks. But by introducing maximum entropy policy, the SAC algorithm, compared with the TD3 algorithm, also excels in exploration ability theoretically.

Algorithm 3 SAC algorithm

Initialize: Critic networks Q_1, Q_2 ; Actor network π ; Target networks Q'_1, Q'_2, π' ; Replay buffer $\mathcal{R}$.

Parameters: Critic networks parameters θ_{Q_1} , θ_{Q_2} ; Actor network parameters θ_π ; Target networks parameters $\theta_{Q'_1} \leftarrow \theta_{Q_1}$, $\theta_{Q'_2} \leftarrow \theta_{Q_2}$, $\theta_{\pi'} \leftarrow \theta_\pi$.

Loop:

Sample action: $a \sim \pi(\cdot|s)$;

Execute a , observe reward r and new state s' ;

Store transition (s, a, r, s') in $\mathcal{R}$;

Update step:

Sample N transitions minibatch (s_i, a_i, r_i, s'_i) from $\mathcal{R}$;

Sample new action $a'_i \sim \pi(\cdot|s'_i)$;

$$V'(s'_i) = \min_{j=1,2} Q'_j(s'_i, a'_i) - \alpha \log \pi(a'_i | s'_i);$$

$$y_i = r_i + \gamma \cdot V'(s'_i);$$

Update Critic networks:

Minimize loss for $Q_{j=1,2}$:

$$L_{j=1,2} = \frac{1}{N} \sum (y_i - Q_{j=1,2}(s_i, a_i))^2;$$

Update Actor network by maximum entropy policy:

Sample new action $\tilde{a}_i \sim \pi(\cdot|s_i)$;

$$Q_{\min} = \min(Q_1(s_i, \tilde{a}_i), Q_2(s_i, \tilde{a}_i));$$

$$\nabla_{\theta_\pi} J(\theta_\pi) = \frac{1}{N} \sum [\nabla_{\theta_\pi} (\alpha \log \pi(\tilde{a}_i | s_i) - Q_{\min})]$$

If automatic α tuning:

Compute target entropy $\mathcal{H}$ and update α to minimize α_{loss} : $\alpha_{\text{loss}} = -\alpha(\log \pi(\tilde{a}_i | s_i) + \mathcal{H})$

Soft update target networks:

$$Q'_{j=1,2}: \theta'_{Q_{j=1,2}} \leftarrow \tau \theta_{Q_{j=1,2}} + (1 - \tau) \theta'_{Q_{j=1,2}};$$

4. NUMERICAL RESULTS

4.1 Numerical Environment

This section conducts spacecraft orbit control simulations for the tracking mission under the non-spherical Earth perturbations and controller noise, using the dynamics model from Section 2.1. The spacecraft orbit control RL training framework is illustrated in Fig. 2. The simulation and algorithm parameters are detailed in Tables 1 and 2 respectively. The training platform of the algorithm and the hardware consists of the Linux 5.15 system with PyTorch 2.6.0, Python 3.10, and Gym 0.26.2, running on an Intel Xeon Gold 5218R CPU and an RTX 3090 GPU.

4.2 Numerical Results and Evaluation

Under the experimental conditions, successful tracking (i.e., mission completion) terminates the experimental episode, so prolonged stable tracking is not visually depicted in the results. However, the trends in velocity error and position error can still demonstrate the algorithms’ control performance by observing their convergence profiles. The velocity error curves, in particular reveal key differences in convergence time and stability among PPO, TD3, and SAC, even when post-success tracking phases are excluded.

After completing the training process, 30 randomly selected episodes are selected for analysis. As shown in Fig. 3(a) and (b), the PPO, TD3, and SAC algorithms effectively meet control accuracy requirements under the conditions of the non-spherical earth perturbation and random controller noise. A comprehensive quantitative evaluation of the three algorithms is provided in Table 3 and 4, including the following key performance metrics: average (AVG)

Table 1. RL hyperparameters

PPO Hyperparameters	Value	TD3 Hyperparameters	Value	SAC Hyperparameters	Value
Total Timesteps	6.4×10^6	Total Timesteps	6.4×10^6	Total Timesteps	5×10^6
Batch Size	1024	Batch Size	1024	Batch Size	1024
Per Update Epochs Number	10	Evaluating Interval	2048	Evaluating Interval	2.5×10^3
Policy Networks	[64,64]	Training Frequency	50	Training Frequency	50
Value Networks	[128,128]	Policy networks	[256,256]	Policy networks	[256,256]
Discount Factor	0.9999	Value networks	[256,256]	Value networks	[256,256]
GAE Factor	0.99	Discount Factor	0.999	Discount Factor	0.99
Actor Learning Rate	1×10^{-3}	Delayed Frequency	1	Learning rate	3×10^{-4}
Critic Learning Rate	1×10^{-4}	Learning Rate	1×10^{-4}	Entropy Coefficient	0.12
Clip Range	0.3	Exploring Noise	0.15	Adaptive Alpha	True
		Explore Noise Decay	0.998		

Table 2. Physical parameters of spacecrafts

Parameter	Value
$\mathbf{r}_b$	$[-5268.945, 3126.25, 3126.25]\text{km}$
$\mathbf{v}_b$	$[0.007, 7.6076, -0.005]\text{km/s}$
m_b	105kg
J_{2b}	$[25, 0, 0; 0, 20, 0; 0, 0, 23] \text{ kg} \cdot \text{m}^2$
$\mathbf{r}_t$	$[-5268.960, 3126.25, 3126.25]\text{km}$
$\mathbf{v}_t$	$[0, 7.6126, 0]\text{km/s}$
m_t	105kg
J_{2t}	$[25, 0, 0; 0, 20, 0; 0, 0, 23] \text{ kg} \cdot \text{m}^2$

velocity tracking error, AVG position tracking error, root mean square error (RMSE) for both velocity and position tracking, convergence time of the control system, and the required moment.

The SAC algorithm achieves the smallest RMSE in tracking error, indicating superior overall accuracy, as further evidenced by its lower average velocity and position tracking errors compared to other algorithms. This advantage stems from the architecture of SAC: while both SAC and TD3 demonstrate stronger exploration capabilities than PPO, SAC's maximum entropy framework further enhances its long-term policy optimization, resulting in greater initial variability but ultimately better accurately than TD3.

In terms of convergence time, the PPO algorithm exhibits the shortest time and lowest variance. Unlike the TD3 and SAC algorithms using state-action value function Q , PPO uses state-value function V to implicitly control action selection by simply fitting the state space. Due to its simpler structure and fewer parameters, the computational efficiency is rather higher under limited data conditions. And the PPO algorithm exploration ability is weaker than TD3 and SAC algorithms, so the fitting ability to disturbance is weaker. Thereby the PPO algorithm exploration policy is firm and instead less affected by the only two disturbances tested in this paper, so that the variance of the convergence time is instead lower.

The maximum and minimum moments in Table 4 are selected based on the Euclidean norm of the moments, and the moment variance is computed as the average variance of all three demensions. As shown in Fig. 3(c) and the column of moment in Table 4, the control moment magnitude and variability during the initial phase exhibit a clear hierarchy (SAC $\approx$ TD3 $>$ PPO). This is consistent with the empirical observation that the greater exploration capacity leads to higher early-stage control moment, with SAC exhibiting the most aggressive exploratory behavior due to

its maximum entropy policy, followed by TD3 and PPO. Notably, TD3 demonstrates the largest moment variance, as SAC's maximum entropy policy enhances environmental adaptability under disturbances, enabling rapid policy adjustment and stabilization, meanwhile PPO's lower sensitivity to disturbances results in more stable control moment.

5. CONCLUSIONS

This paper investigates close-range spacecraft orbit tracking under complex disturbance conditions by using DRL algorithm, demonstrating that the PPO, SAC, and TD3 algorithms achieve high-precision tracking despite the non-spherical earth perturbation and random controller noise. The numerical simulations demonstrate that the computational efficiency and stable moment output of PPO make PPO preferable for less disturbances environments requiring rapid stabilization, while TD3 and SAC's double-network architecture render it superior for precision tracking in dynamically complex environments and SAC performs better than TD3. Future research could focus on enhancing the exploration ability of PPO, or sample efficiency in TD3 and SAC to broaden reinforcement learning applications in more space missions.

REFERENCES

- Barto, A.G., Sutton, R.S., and Anderson, C.W. (1983). Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-13(5), 834–846. doi: 10.1109/TSMC.1983.6313077.
- Cheng, L., Wang, Z., Jiang, F., and Zhou, C. (2018). Real-time optimal control for spacecraft orbit transfer via multiscale deep neural networks. *IEEE Transactions on Aerospace and Electronic Systems*, 55(5), 2436–2450.
- Fujimoto, S., Hoof, H., and Meger, D. (2018). Addressing Function Approximation Error in Actor-Critic Methods. In *Proceedings of the 35th International Conference on Machine Learning*, 1587–1596. PMLR.
- Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., Kumar, V., Zhu, H., Gupta, A., Abbeel, P., and Levine, S. (2019). Soft Actor-Critic Algorithms and Applications. doi:10.48550/arXiv.1812.05905.
- Kakade, S. and Langford, J. (2002). Approximately optimal approximate reinforcement learning. In *Proceedings of the Nineteenth International Conference on Machine Learning*, ICML '02, 267–274. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.

Table 3. Tracking error statistical results of spacecraft orbit control

	AVG Velocity Error(km)			AVG Position Error(km)			RMSE(km ²)	
	X	Y	Z	X	Y	Z	Velocity	Position
PPO	0.0036	0.0021	0.0017	0.0107	0.0032	0.0027	0.0030	0.0050
TD3	0.0033	0.0021	0.0020	0.0113	0.0039	0.0044	0.0028	0.0052
SAC	0.0026	0.0017	0.0019	0.0103	0.0027	0.0048	0.0025	0.0046

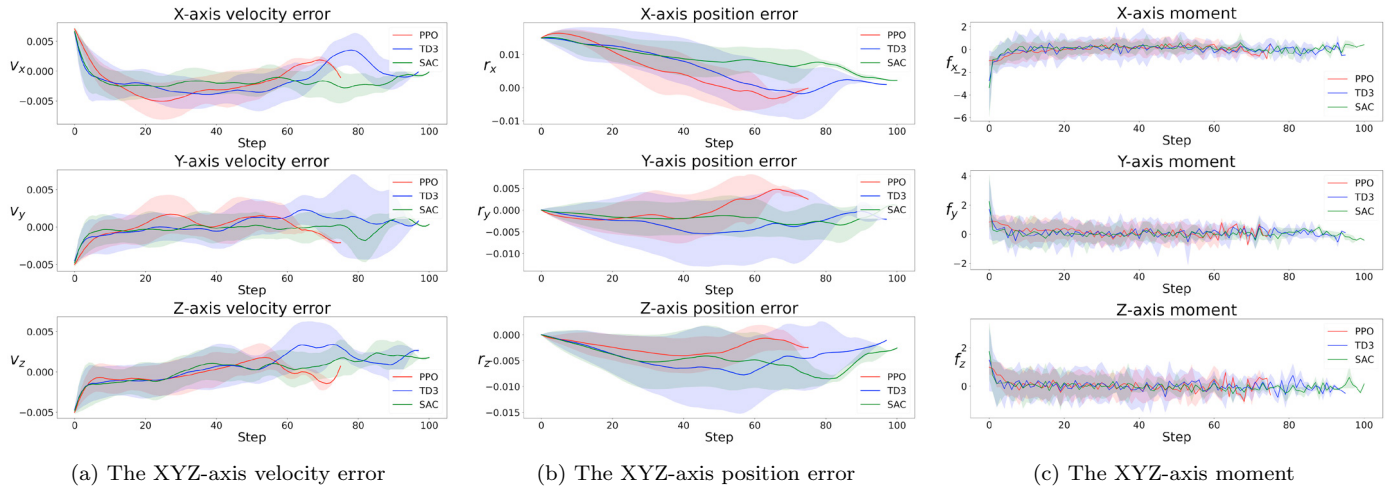


Fig. 3. The simulation result of spacecraft orbit control under external disturbances using PPO, TD3, and SAC

Table 4. Convergence and moment statistical results of spacecraft orbit control

	Settling Time(s)		Moment(N · m)		
	Average	Variance	Max	Min	Variance
PPO	3.99	177.36	1.6389	0.1023	0.5913
TD3	4.39	563.49	3.4574	0.0753	1.0809
SAC	5.08	667.69	4.3727	0.0867	0.7410

- Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D. (2019). Continuous control with deep reinforcement learning. URL <https://arxiv.org/abs/1509.02971>.
- Schulman, J., Levine, S., Abbeel, P., Jordan, M., and Moritz, P. (2015). Trust Region Policy Optimization. In *Proceedings of the 32nd International Conference on Machine Learning*, 1889–1897. PMLR.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal Policy Optimization Algorithms. doi:10.48550/arXiv.1707.06347.
- Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., and Lanctot, M. (2016). Mastering the game of Go with deep neural networks and tree search. *nature*, 529(7587), 484–489.
- Sutton, R.S. and Barto, A.G. (1999). Reinforcement learning: An introduction. *Robotica*, 17(2), 229–235. doi:10.1017/S0263574799271172.
- Tang, Z., Shao, K., Zhao, D., and Zhu, Y. (2017). Recent progress of deep reinforcement learning: From AlphaGo to AlphaGo Zero. *Control Theory & Applications*, 34(12), 1529–1546.
- Tao, W., Kai, Z., Xinbo, W., and Xianfeng, Z. (2022). A path planning method for geo satellite on-orbit service. *Aerospace Control*, 40(6), 11–16.
- Wang, B., Chen, W., Zhang, B., Shi, P., and Zhang, H. (2023). A Nonlinear Observer-Based Approach to Robust Cooperative Tracking for Heterogeneous Spacecraft

Attitude Control and Formation Applications. *IEEE Transactions on Automatic Control*, 68(1), 400–407. doi:10.1109/TAC.2022.3143082.

- Yadava, D., Hosangadi, R., Krishna, S., Paliwal, P., and Jain, A. (2018). Attitude control of a nanosatellite system using reinforcement learning and neural networks. In *2018 IEEE Aerospace Conference*, 1–8. IEEE.
- Zhao, F.Y., Xu, R., and Cui, P.Y. (2015). Heuristic mission planning approach for deep space explorer. *Yuhang Xuebao/Journal of Astronautics*, 36(5), 496–503. doi:10.3873/j.issn.1000-1328.2015.05.002.
- Zhao, S. (2025). *Mathematical Foundations of Reinforcement Learning*. Springer Nature Singapore, Singapore. doi:10.1007/978-981-97-3944-8.